SAPIENTIA
ERDÉLYI MAGYAR
TUDOMÁNYEGYETEM

Sapientia Erdélyi Magyar Tudományegyetem

Marosvásárhelyi Kar

Számítástechnika szak

3D nyomtatási folyamat monitorizálás és hibadetektálás mesterséges intelligenciával

Unit Teszt Dokumentáció

Készítette: Borbáth Mátyás-Levente

2026

Tartalomjegyzék

1. Áttekintés **3**

2. Általános tesztelési stratégia **3**

3. `test_judge.py` - a Judge szolgáltatás tesztjei **4**

3.1. Időbélyeg-generálás (`TestNowIso`) 4

3.2. Base64 dekódolás (`TestSafeB64Decode`) 5

3.3. HTTP végpontok (`TestHandlerGet` és `TestHandlerPost`) 5

4. `test_alert_manager.py` - az Alert Manager tesztjei **5**

4.1. Időzóna-konverzió (`TestHumanTime`) 5

4.2. Hűtési logika (`TestIsOnCooldown` és `TestSetCooldown`) 6

4.3. E-mail küldés (`TestSendEmail`) 6

4.4. Detekciós esemény feldolgozás (`TestHandleDetection`) 6

4.5. Költségvetési figyelmeztetés (`TestHandleBudgetAlert`) 7

5. `test_dispatcher.py` - a Dispatcher tesztjei **7**

5.1. OAuth2 token kezelés (`TestGetAuthHeader`) 7

5.2. Vertex AI kérés küldése (`TestDispatchFrame`) 7

6. `test_frame_extractor.py` - a Frame Extractor tesztjei **8**

6.1. Képkocka-rögzítési ciklus (`TestCaptureLoop`) 8

6.2. Szálindítás és konfiguráció elemzése (`TestMainParsing`) 8

6.3. Időzítés és kódolás 8

7. CI/CD integráció **9**

7.1. Egységtesztek futtatása (`python-tests.yml`) 9

7.2. Infrastruktúra-ellenőrzés és -telepítés (`terraform.yml`) 9

7.3. Judge konténer build és push (`docker-judge.yml`) 10

7.4. Dolgozat fordítása (`compileLatex.yml`) 10

8. Összefoglalás **11**

1. Áttekintés

A rendszer négy fő mikroszolgáltatásból áll, és mindegyikhez készültek egységtesztek. A tesztelés célja az volt, hogy a szolgáltatások üzleti logikáját valódi felhős infrastruktúra nélkül is ellenőrizni lehessen, hiszen egy Vertex AI-végpont vagy Firestore-adatbázis felállítása minden egyes tesztfuttatáshoz nem kivitelezhető.

Az egységtesztek négy fájlba vannak szervezve, amelyek rendre a Judge, az Alert Manager, a Dispatcher és a Frame Extractor szolgáltatásokat fedik le. A tesztelési keretrendszer a `pytest`, a lefedettség mérésére pedig a `pytest-cov` csomag szolgál. A projekt egy 90%-os kódlefedettségi küszöböt ír elő, amelyet mind a pre-commit hook, mind a GitHub Actions CI/CD folyamat kikényszerít, vagyis ha ez az arány nem teljesül, a build elbukik és a kód nem kerülhet be a főágba.

1. táblázat. Tesztfájlok és a lefedett szolgáltatások

Tesztfájl	**Szolgáltatás**	**Lefedett funkciók**
`test_judge.py`	Judge (Vertex AI)	időbélyeg-generálás, Base64 dekódolás, HTTP végpontok
`test_alert_manager.py`	Alert Manager (CF)	időzóna-konverzió, várakozási logika, e-mail küldés, eseményfeldolgozás
`test_dispatcher.py`	Dispatcher (CF)	OAuth2 token kezelés, Vertex AI kérés továbbítás
`test_frame_extractor.py`	Frame Extractor (CR)	képkocka-rögzítés, JPEG kódolás, szálindítás

2. Általános tesztelési stratégia

Az egyik legnagyobb kihívás a tesztek megírásánál az volt, hogy a szolgáltatások erősen függnek külső könyvtáraktól és GCP-infrastruktúrától: a Judge például az `ultralytics` YOLO-csomagot és a Google Cloud Pub/Sub klienst használja, a Frame Extractor pedig az `OpenCV`-t. Ezeket valódi környezetben futtatni tesztelési célból túlságosan erőforrás-igényes lett volna.

A megoldás az volt, hogy ezeket a függőségeket a Python `sys.modules` szótárán keresztül, a tényleges modulok betöltése *előtt* `MagicMock` objektumokkal helyettesítettük. Így a tesztek nem igényelnek valódi GCP-hitelesítő adatokat, GPU-t, kamerát vagy internet-kapcsolatot, bármilyen

gépen, másodpercek alatt lefutnak.

A szolgáltatások forráskódját az `importlib` segítségével töltjük be fájlútvonal alapján, nem pedig normál `import` utasítással, mivel a szolgáltatások nem Python-csomagként vannak strukturálva. A Cloud Function belépési pontokat a `@functions_framework.cloud_event` dekorátor védi, amelyet szintén semlegesítünk egy egyszerű identity-függvénnyel, hogy a függvények hagyományos Python-függvényekként hívhatók legyenek a tesztekből.

A Frame Extractor esetében egy külön problémát jelentett a `capture_loop()` végtelen ciklusa, amelyet normál körülmények között csak a folyamat leállítása szakít meg. Erre egy ún. sentinel-kivétel technikát alkalmaztunk: a `cv2.VideoCapture` mock objektumot úgy állítottuk be, hogy a második hívásnál egy `_StopLoop` kivételt dobjon, amelyet a `pytest.raises()` elfog, és a teszt szabályosan befejeződik.

Listing 1. Sentinel-kivétel a végtelen capture ciklus megtöréséhez

```
class _StopLoop(Exception):
    pass

fe.cv2.VideoCapture.side_effect = [cap, _StopLoop]

with pytest.raises(_StopLoop):
    fe.capture_loop("rtsp://cam/stream", "cam1")
```

3. `test_judge.py` - a Judge szolgáltatás tesztjei

A Judge egy Vertex AI-on futó konténer, amely a beérkező kameraképeken YOLOv8x modellel végez objektumdetektálást, majd az eredményt a `detections-out` Pub/Sub témára teszi közzé. A tesztfájl négy osztályba szervezi az ellenőrzéseket, amelyek a két segédfüggvényt és a HTTP kezelőt fedik le.

3.1. Időbélyeg-generálás (`TestNowIso`)

A `now_iso()` függvény minden detekciós üzenetbe belekerülő UTC időbélyeget állít elő ISO 8601 formátumban. A tesztek ellenőrzik, hogy a visszatérési érték valóban szöveg típusú, hogy az `fromisoformat()` sikeresen értelmezi, hogy az időzóna UTC, és hogy két egymást követő hívás eredménye nem csökkenő sorrendű, vagyis az időbélyeg nem „megy vissza az időben".

3.2. Base64 dekódolás (`TestSafeB64Decode`)

A Pub/Sub üzenetekben a képadatok Base64 kódolással érkeznek, és a gyakorlatban előfordulhat, hogy a kódolt sztringből hiányoznak a kitöltő = karakterek, vagy szóközök kerülnek bele átvitel közben. A `_safe_b64decode()` függvény ezeket a hibákat automatikusan javítja.

A tesztek lefedik a normál, kitöltött bemenetet, az egy és két hiányzó = karakteres eseteket, a szóközök eltávolítását, valamint azt is, hogy üres, `None` vagy kizárólag szóközöket tartalmazó bemenetre a függvény `ValueError` kivételt dob, mert egy üres kép adatai értelmetlen bemenetet jelentenének a YOLO-modell számára.

3.3. HTTP végpontok (`TestHandlerGet` és `TestHandlerPost`)

A Judge konténer egy egyszerű HTTP szervert futtat. A GET kérések három egészségügyi ellenőrzési útvonalat (`/healthz`, `/health`, `/`) kezelnek 200-as válasszal, az ismeretlen útvonalak 404-et kapnak. A POST `/predict` végpont kétféle kérésformátumot fogad el: a Vertex AI saját *instances* burkolatát és a közvetlen Pub/Sub üzenetformátumot.

A tesztek ellenőrzik a helyes HTTP státuszkódokat, azt, hogy sikeres inferencia esetén a `publish()` pontosan egyszer hívódik, és azt is, hogy ha az `cv2.imdecode()` `None`-t ad vissza (sérült kép), a végpont 500-as hibával válaszol ahelyett, hogy kivételt propagálna.

4. `test_alert_manager.py` - az Alert Manager tesztjei

Az Alert Manager Cloud Function a `detections-out` Pub/Sub eseményeket dolgozza fel: magas súlyosságú detekció esetén e-mailt küld az üzemeltetőnek, minden detekciót Firestore-ba ment, és kezeli a GCP-költségvetési figyelmeztetéseket is. Ez a rendszer legösszetettebb üzleti logikáját tartalmazó komponense, ezért a legtöbb teszteset is ehhez tartozik, összesen 29 darab, hat osztályba szervezve.

4.1. Időzóna-konverzió (`TestHumanTime`)

Az e-mail értesítésekben a detekció időpontja olvasható formátumban jelenik meg, Bukarest helyi ideje szerint (Europe/Bucharest, UTC+2 nyáron). A `human_time()` függvény végzi ezt az átalakítást. A tesztek ellenőrzik az UTC → helyi idő konverziót (17:38 UTC esetén 19:38-at kell

kapnunk), a dátum részeinek meglétét a kimenetben, és azt, hogy érvénytelen bemenetre a függvény az eredeti sztringet adja vissza hiba helyett.

4.2. Várakozási logika (`TestIsOnCooldown` és `TestSetCooldown`)

Annak érdekében, hogy egy folyamatosan fennálló hiba ne árassza el az üzemeltetőt e-mailekkel, a rendszer kameránként 60 másodperces várakozási időszakot alkalmaz. Az utolsó küldés időpontját a Firestore tárolja `last_sent` mezőként, kamera-azonosítóhoz rendelve.

Az `is_on_cooldown()` tesztjei lefedik azt az esetet, amikor a dokumentum még nem létezik (nincs várakozási idő), amikor 10 másodperccel ezelőtt küldtünk (aktív várakozási idő), 120 másodperccel ezelőtt (lejárt), és a határesetet: pontosan 59 másodperccel ezelőtt küldtünk, ami még mindig aktívnak számít, mivel az intervallum zárt.

A `set_cooldown()` tesztjei ellenőrzik, hogy a Firestore `set()` művelete meghívódik, hogy az elmentett időbélyeg érvényes ISO 8601 formátumú, és hogy a dokumentum kulcsa a kamera azonosítója.

4.3. E-mail küldés (`TestSendEmail`)

A `send_email()` az `smtplib.SMTP_SSL` osztályon keresztül a Gmail SMTP szerverhez csatlakozik (port 465), és HTML formátumú e-mailt küld. A tesztek ellenőrzik a szerver és port helyes értékét, a bejelentkezéshez használt hitelesítő adatokat, a tárgy megjelenését a nyers üzenetben, és azt is, hogy SMTP kapcsolódási hiba esetén a kivétel nem propagálódik tovább, egy e-mail küldési hiba nem akadályozhatja meg az egyébként sikeres detekció mentését Firestore-ba.

4.4. Detekciós esemény feldolgozás (`TestHandleDetection`)

A `handle_detection()` a rendszer legösszetettebb függvénye: szűri a konfidencia-küszöb alatti detekciókát, megkülönbözteti a magas (`spaghetti`, `not_sticking`, `layer_shift`, `warping`) és alacsony súlyosságú hibaosztályokat, kezeli a várakozási időszakot, és Firestore-ba írja az eseményeket. Ennek megfelelően ez az osztály tartalmazza a legtöbb tesztesetet.

A tesztek ellenőrzik, hogy 0,10-es konfidenciájú detekció nem kerül mentésre és nem vált ki e-mailt, hogy 0,90-es konfidenciánál a Firestore `add()` pontosan egyszer hívódik, hogy alacsony súlyosságú osztály (pl. `stringing`) esetén nem megy e-mail, hogy magas súlyosságú osztály (pl. `warping`) esetén megy, és hogy a tárgy tartalmazza a kamera azonosítóját. Külön tesztesetek fedik

le az összes magas súlyosságú osztályt egyenként, a pontos küszöbértéket (0,35 éppen átmegy), a küszöb alatti értéket (0,3499 kiszűrődik), az üres detekciós listát, és a hibás Base64 adatot tartalmazó eseményt, amely nem propagálhat kivételt.

4.5. Költségvetési figyelmeztetés (`TestHandleBudgetAlert`)

A `handle_budget_alert()` a GCP Budget API értesítéseit dolgozza fel, Firestore-ba menti az aktuális összeget és a küszöbértéket, és várakozási időn kívül e-mailt küld. A tesztek ellenőrzik a Firestore dokumentum mezőit és értékeit, az e-mail küldését várakozási időn kívül, az e-mail elmaradását várakozási időn belül, és a hibás bemenetre való robusztus viselkedést.

5. `test_dispatcher.py` - a Dispatcher tesztjei

A Dispatcher Cloud Function a `frames-in` Pub/Sub témáról érkező képkockákat a Vertex AI Online Prediction végpontjára továbbítja. Mivel a Vertex AI csak érvényes OAuth2 tokent tartalmazó kéréseket fogad el, a Dispatcher maga kezeli a hitelesítési token frissítését.

5.1. OAuth2 token kezelés (`TestGetAuthHeader`)

A `_get_auth_header()` egy `Authorization: Bearer` fejlécet ad vissza, amelyet a Vertex AI kérésbe kell beilleszteni. Ha a tárolt token érvényes, nem kell frissíteni; ha lejárt, a `refresh()` automatikusan meghívódik. A tesztek mindhárom esetet lefedik: érvényes token esetén nem hívódik `refresh()`, lejárt token esetén pontosan egyszer hívódik, és a visszaadott fejléc tartalmazza az `Authorization: Bearer` és `Content-Type: application/json` mezőket.

5.2. Vertex AI kérés küldése (`TestDispatchFrame`)

A `dispatch_frame()` dekódolja a Pub/Sub üzenetet, és összeállítja a Vertex AI *instances* burkolatot, amely tartalmazza a képadatot (`data_b64`), a kamera azonosítóját és a szekvenciaszámot.

Fontos viselkedés, hogy ha a payload nem tartalmaz képadatot, a függvény nem küld HTTP kérést. HTTP hiba esetén viszont a kivétel szándékosan propagálódik, hogy a Pub/Sub automatikus újrapróbálkozási mechanizmusa aktiválódjon, ez a megbízhatóság szempontjából kulcsfontosságú tervezési döntés. A tesztek azt is ellenőrzik, hogy az `image_b64` mező visszaesési lehetőségként elfogadott, és a kérésbe `data_b64` kulcsként kerül be.

6. `test_frame_extractor.py` - a Frame Extractor tesztjei

A Frame Extractor Cloud Run konténer több RTSP kamerafolyamot olvas párhuzamosan, a képkockákat JPEG formátumba tömöríti, Base64-gyel kódolja, és közzéteszi a `frames-in` Pub/Sub témára.

6.1. Képkocka-rögzítési ciklus (`TestCaptureLoop`)

A `capture_loop()` egy adott kamerafolyamot folyamatosan olvas. Ha a stream nem nyílik meg, 5 másodpercet vár és újrapróbálkozik; ha egy képkocka olvasása sikertelen, felszabadítja az erőforrásokat (`cap.release()`) és újranyit.

A tesztek ellenőrzik az újrapróbálkozást és az 5 másodperces várakozást, a `release()` hívását sikertelen olvasás esetén, a Pub/Sub `publish()` meghívását sikeres képkocka esetén, a közzétett JSON kötelező mezőit (`camera_id`, `seq`, `ts`, `data_b64`), és azt, hogy a szekvenciaszám három egymást követő képkockánál 0, 1, 2 értékeket vesz fel.

6.2. Szálindítás és konfiguráció elemzése (`TestMainParsing`)

A `main()` beolvassa az RTSP URL-eket és kamera-azonosítókat a környezeti változókból, és minden kamerához külön szálat indít. Ha az RTSP URL-ek listája üres, `RuntimeError` keletkezik, mert a konténernek nincs mit rögzítenie. Ha kevesebb azonosítót adunk meg, mint ahány URL van, a hiányzók automatikusan `cam1`, `cam2` stb. neveket kapnak.

6.3. Időzítés és kódolás

A `CAPTURE_FPS` paraméter szabályozza a rögzítés frekvenciáját. Ha egy képkocka feldolgozása gyorsabb az intervallumnál, a ciklus `time.sleep()`-pel várja be a maradék időt; ha lassabb, nincs lassítási `sleep()`, a ciklus azonnal folytatódik.

A kódolási tesztek ellenőrzik, hogy a `cv2.resize()` a konfigurált `FRAME_WIDTH` és `FRAME_HEIGHT` értékekkel hívódik, és hogy a `cv2.imencode()` `.jpg` formátumot és a konfigurált `JPEG_QUALITY` értéket alkalmazza.

7. CI/CD integráció

A projekt négy GitHub Actions workflow-t tartalmaz, amelyek különböző eseményekre aktiválódnak. Mindegyik workflow csak azokat a fájlokat figyeli, amelyek az adott feladathoz valóban relevánsak, ezt az útvonalszűrés (`paths`) biztosítja, így például egy dokumentációs módosítás nem indít el feleslegesen Terraform-futtatást.

7.1. Egységtesztek futtatása (`python-tests.yml`)

Ez a workflow az egységtesztek szempontjából a legfontosabb. Minden olyan `push` és `pull_request` eseményre aktiválódik a `main` ágon, amely a `terraform_v2/services/` vagy a `tests/` mappát érinti.

A futtatás lépései a következők: a Python 3.12-es verzióját állítja be pip-gyorsítótárral, telepíti a `requirements-test.txt` függőségeit, majd lefuttatja a teljes tesztcsomagot a következő paraméterekkel:

Listing 2. A tesztfuttatás parancsa a CI-ban

```
python -m pytest tests/ -v \
  --tb=short \
  --cov=terraform_v2/services \
  --cov-report=term-missing \
  --cov-report=xml \
  --cov-fail-under=90
```

A `-cov-fail-under=90` kapcsoló azt jelenti, hogy ha a kódlefedettség 90% alá esik, a build elbukik, és a kód nem kerülhet be a főágba. A lefedettségi jelentés XML formátumban is előáll (`coverage.xml`), amelyet a workflow 14 napig tárol artifact-ként, ez lehetővé teszi a lefedettség történetének utólagos vizsgálatát.

7.2. Infrastruktúra-ellenőrzés és -telepítés (`terraform.yml`)

Ez a workflow a Terraform-konfigurációt validálja és alkalmazza. Négy egymásra épülő feladatból áll.

Az első feladat a **lint**, amely két eszközt futtat egymás után. A `terraform fmt -check` ellenőrzi, hogy minden `.tf` fájl a Terraform által előírt formátumkövetelményeknek megfelelően van-e tagolva. Ezután a `tflint` elemzi a konfigurációt potenciális hibák és rossz gyakorlatok

szempontjából, például elavult erőforrás-attribútumok vagy nem hatékony változóhasználat kapcsán. Fontos megjegyezni, hogy ezek *nem* egységtesztek: nem az alkalmazás logikáját, hanem az infrastruktúra-leíró kód minőségét és stílusát ellenőrzik.

A második feladat a **Checkov biztonsági vizsgálat**, amely a Terraform-konfigurációt ismert biztonsági és megfelelőségi problémák szempontjából elemzi, például nyilvánosan elérhető tárolóvödrök, titkosítás hiánya vagy nem megfelelő jogosultságok. Ez sem egységteszt, hanem egy infrastruktúra-biztonsági szkennelés; a `soft_fail: true` beállítás miatt figyelmeztetéseket ad, de nem akasztja meg a pipeline-t.

Ha a lint és a Checkov sikeresen lefut, a **Terraform Plan** feladat GCP-hitelesítéssel (`GCP_SA_KEY` titok) elvégzi az inicializálást és a tervkészítést. Pull Request esetén a terv kimenetét automatikusan kommentként fűzi a PR-hez, hogy a felülvizsgálók lássák, milyen változások kerülnének élesbe. Ha a terv hibás, a feladat sikertelennek minősül.

A **Terraform Apply** feladat kizárólag akkor fut, ha a változtatás a `main` ágra érkezik közvetlen `push` esemény formájában (tehát nem PR-ból). Ez biztosítja, hogy az infrastruktúra csak felülvizsgált és jóváhagyott kód alapján frissüljön.

7.3. Judge konténer build és push (`docker-judge.yml`)

Ha a Judge szolgáltatás kódja megváltozik, ez a workflow letölti a YOLO-modell súlyait (`best.pt`) a GCS-vödörből, majd felépíti a Docker-képet. A kép két taggel kerül feltöltésre az Artifact Registry-be: a commit SHA-val (a pontos verzió visszakereshetőségéhez) és a `latest` taggel. A push csak `main` ágra érkező közvetlen kommit esetén történik meg, Pull Request esetén csak a build lép, a push nem.

7.4. Dolgozat fordítása (`compileLatex.yml`)

Ha a `docs/latex/` mappában változás történik, a workflow LaTeX-csomagokat telepít, háromszor futtatja a `pdflatex`-et (a kereszthivatkozások feloldásához), Ghostscript-tel tömöríti a PDF-et, majd visszacommitálja a lefordított dokumentumot a repóba. A commit üzenetébe bekerül a `[skip ci]` jelzés, hogy ez a commit ne indítson el újabb fordítási ciklust.

8. Összefoglalás

A projekt összesen 70 egységtesztet tartalmaz, amelyek a négy mikroszolgáltatás üzleti logikájának kritikus részeit fedik le. A tesztek kizárólag mockolással dolgoznak, így nem igényelnek valódi GCP-infrastruktúrát, és helyi gépen is futtathatók. A 90%-os lefedettségi küszöb betartása folyamatosan ellenőrzött, és a CI/CD folyamat részeként minden változtatásnál automatikusan lefut.

2. táblázat. Tesztosztályok és tesztesetek száma szolgáltatásonként

Tesztfájl	**Tesztosztály**	**Tesztek száma**
`test_judge.py`	`TestNowIso`	4
	`TestSafeB64Decode`	7
	`TestHandlerGet`	4
	`TestHandlerPost`	5
`test_alert_manager.py`	`TestHumanTime`	3
	`TestIsOnCooldown`	4
	`TestSetCooldown`	3
	`TestSendEmail`	4
	`TestHandleDetection`	11
	`TestHandleBudgetAlert`	4
`test_dispatcher.py`	`TestGetAuthHeader`	3
	`TestDispatchFrame`	5
`test_frame_extractor.py`	`TestCaptureLoop`	5
	`TestMainParsing`	4
	`TestCaptureLoopTiming`	2
	`TestCaptureLoopEncoding`	2
Összesen		**70**